CS 458/558 Introduction to Computer Security, Spring 2025
Programming Assignment 2

Out: 3/25/2025 Tue.
Due: 4/9/2025 Wed. 23:59:59

In this task, you will work with a program that contains a buffer overflow vulnerability. Your objective is to write programs to generate attack string to exploit this vulnerability. Additionally, you are also required to exploit the buffer overflow to execute provided shellcode and launch a shell.

Setup

You are provided with the vulnerable code `vuln_program.c`. You can download this code from Brightspace assignments section.
This program should be compiled with the following command on your CS department machine(remote or not). (Be aware, for the following exploitation, run it on department machine to make it coherent for the grading. Failed to do so may have to schedule an appointment with TA for a code demo).

```
gcc -fno-stack-protector -z execstack -fno-pie -no-pie -m32 \
    -O0 -g vuln_program.c -o vuln_program
```

The above command will use GNU C compiler to compile the vulnerable code with options(in the order they applied) no stack protector, executable stack, no position independent code, Intel x86 32 bits mode, no optimization applied and GDB debugger information enabled. The output is the vulnerable program `vuln_program`. You may see a compiler warning about the implicit declaration of `gets`. This is because `gets` has been deprecated due to security risks. However, we intentionally use it in our vulnerable program.

Exploitation

The goal of this exploitation is to write your own program and generate an attack string and store it in a file named `attack_string`.
For this part, you are required to launch an exploitation that without providing a correct password for the vulnerable program, the program can still output `"You have entered the correct passwd"`.
To successfully fulfill this task, you have to

1. Compile the the `vuln_program` using the commands provided above.

2. Use a binary analyzing tool to get the address of `target` function. Tools like nm, readelf and objdump can fulfill this job. Read the corresponding manual to figure out how to achieve this.

3. After got the `target` function address, now you have to construct the attack string using your program. And store it in the `attack_string` file. You program should take the target

function address as the input. For example if 0xdeadbeef is the target function address, and your code is written in python, then running `"python3 attack.py deadbeef"` will create an string stored in `attack_string` file to be further used to exploit the vulnerability in the vulnerable program.

4. To verify your exploitation, run the program with command `./vuln_program < attack_string` This should gives you the expected output `"You have entered the correct passwd"`. If you failed the exploitation, the program usually give you `"Segmentation fault"` error. Then, try to fix the bug in your string generation program, and make it work.

Despite the vulnerable program being written in C, you may use any programming language you are comfortable with to generate the attack string. Make sure the department machines(Lab G7) or remote machines support your language choices. The assignment will be graded on these machines. Contact the TA if your language is not supported by the department machines.

Construct the attack string

The following will try to help you understand how to construct an attack string.

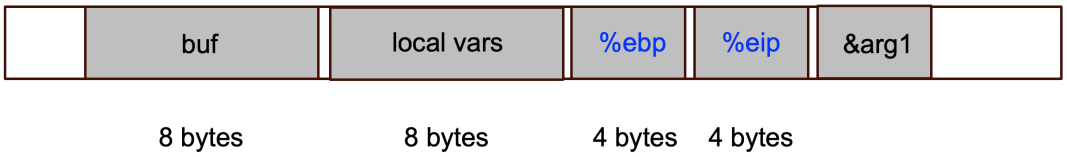


Figure 1: Stack layout of vulnerable program.

Figure1 shows the stack layout while the `prompt` function invoking. The goal is to overwrite the buffer until return address(%eip) on the stack has contains the target function address. Based on this, construct you attack string carefully. One thing to be aware is the address in little-endian format. For example, if target address is "0xdeadbeef", then the bytes of return address at RA will be RA[0]:ef, RA[1]:be, RA[2]:ad, RA[3]:de.

Launch a shell

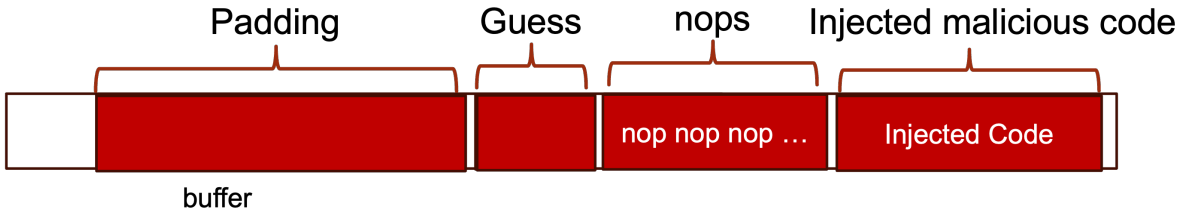


Figure 2: Stack layout of launching shellcode.

As shown in the Figure2, for this 2nd part, you are going to reuse the attack program to generate another attack payload for this shellcode exploitation.

1. Overwrite the buffer with padding.
2. Overwrite the return address(%eip) on the stack with a guessed address that **probably** will jump to the injected malicious code.
3. nops(0x90) can be filled in the between the return address and injected malicious code to increase the chance that injected malicious code will be executed.
4. The shellcode then provided as payload at the end of the overwrite.

The shellcode that is used to launch a shell is provided as following:

```
"\x31\xc0\x31\xdb\xb0\x06\xcd\x80\x53\x68/tty\x68/dev\x89\xe3\x31\xc9
\x66\xb9\x12\x27\xb0\x05\xcd\x80\x31\xc0\x50\x68//sh\x68/bin\x89\xe3
\x50\x53\x89\xe1\x99\xb0\x0b\xcd\x80"
```

Be aware, due to the space limitation, I have to break this shellcode into 3 lines. If you copy this code into your program, make sure it's a **single line** string, which does not contain any newline characters.
Provide the argument as "shellcode" to the attack program will generate the shellcode attack payload. For example, if your code is written in python, run your program as `"python3 attack.py shellcode"`. The output of your program should be a file named `"shell_string"` which stores the attack payload for launching the shellcode.
To verify your shellcode payload, you have to run this command:
`setarch 'uname -m' -R ./vuln_program < shell_string`
The reason we have to do this is the department machines have already enabled address layout randomization(ASLR) protection, which will make the program have a different stack address each time you run it. This can fail your exploitation nearly all the time. The above command is a workaround that we can apply to the vulnerable program to disable the ASLR for this running round.
A successful exploit will show you a `$` sign in a new line, which basically launches a new shell program. To get back to the previous shell, run `exit`. A failed exploit will probably give you "Segmentation fault". To debug the payload generated for the vulnerable program, you could use GDB to help you. Launch the vulnerable program with `gdb ./vuln_program`. Then you can add breakpoints inside the program by using `break`. Run the program with attack payload using `run < shell_string`. An detailed introduction to GDB can be founded here. Please keep in mind that the stack address of the vulnerable program may differ when running it in GDB compared to running it directly from the command line.

Log and submit your work

Log your work: besides the source files of your assignment, you must also include a README file which minimally contains your name, B-number, programming language you used and how to compile/execute your program. Additionally, it can contain the following:

- The status of your program (especially, if not fully complete).
- A description of how your code works, if that is not completely clear by reading the code (note that this should not be necessary, ideally your code should be self-documenting).

- Any other material you believe is relevant to the grading of your project.

Compress the files: compress your README file, all the source files in the same folder, and any additional files you add into a ZIP file. Name the ZIP file based on your BU email ID. For example, if your BU email is "abc@binghamton.edu", then the zip file should be "proj2_abc.zip".

Submission: submit the ZIP file to Brightspace before the deadline.

Lastly but not the least, stick to the collaboration policy stated in the syllabus: you may discuss with your fellow students, but code should absolutely be kept private. Any violation to the Academic Honesty Policy and University Academic Policy, will result 0 for the work.